

TDP019 Projekt: Datorspråk

Språkdokumentation: CnAkE

Författare

Emil Peiponen: emipe528@student.liu.se

Abdullah Qadir: abdqa105@student.liu.se

1 Revisionshistorik

Ver.	Revisionsbeskrivning	Datum
1.0	Första versionen	25-05-13
1.1	Andra versionen	26-05-12

2 Inledning

Detta dokument är en del av ett projekt på programmet för Innovativ Programmering (IP) inom kursen TDP019 Projekt: datorspråk, som ges under andra terminen av utbildningen.

Syftet med projektet är att ge oss praktisk erfarenhet av att designa och implementera ett eget programmeringsspråk från grunden. Det är baserat på den teoretiska grund vi fått i tidigare kurser, som TDP007 Konstruktion av datorspråk samt den fördjupning vi får under denna kurs.

I detta projekt har vi skapat ett eget programmeringsspråk med namnet av CnAkE. Språket är inspirerat av både Python och C++ med målet att kombinera det bästa från båda språken, till exempel enkelheten och läsbarheten från Python samt struktur och flexibilitet från C++. Vi har lagt särskild vikt vid att förenkla vanliga frustrationsmoment hos programmerare, exempelvis problem med indentering och brist på tydlighet i vissa kontrollstrukturer.

3 Användarhandledning

Vårt programmeringsspråk, CnAkE, är ett lättanvänt språk där målgruppen, som är nya programmerare, kan testa sin grundläggande kod. Eftersom språket är väldigt simpelt jämfört med andra språk, så finns det exempelvis inte mer avancerade system som klasser eller en mängd basfunktioner. Detta gör att språket blir mindre överväldigande för nya programmerare. Dessa kan vara elever i grundskolan eller gymnasiet som gillar att skapa och få snabb feedback på om deras kod fungerar.

Språket körs i intepretatorläge i Ruby, där du skriver i själva kodprogrammet. Du kommer också att behöva filen CnAkEparse.rb. Intepretatorläget startas genom att först ladda ned Ruby på din valda dator, och sedan skriva "irb" i terminalen. När du är i läget skall "require './CnAkEparser.rb'" och "CnAkE.new.slither" skrivas, där du nu är öppen att skriva i CnAkE språket.

3.1 Variabler

En bra början för ett språk är variabler. Variabler skapade i CnAkE börjas först med vilken slags datatyp den är, som t.ex 'int' och 'chr'. Variabelnamnet måste innehålla minst en bokstav för att fungera. Sedan följer en tilldelning beroende på vilken datatyp variabeln är.

Ex.

```
int arg = 2;
```

```
str x = "hello";  
  
chr a = 'a';
```

Kodsnitt 1: Variabel

3.2 Output

Att skriva ut ett tal eller en variabel kan vara en stor hjälp. I CnAkE använder du basfunktionen "print" för att skriva ut det du vill.

Ex.

```
print 2;  
  
print x;  
  
print "hello";
```

Kodsnitt 2: Utskrift

3.3 Aritmetik

I CnAkE kan användaren göra aritmetik. Bara heltal fungerar där man skriver raa nummer, medans med variabler med datatypen 'pnt' kan decimaltal även användas.

Ex.

```
2+2
```

Kodsnitt 3: Aritmetik

3.4 Loopar

Som andra språk finns det en rätt mängd olika loopar som språket innehåller. For Loop är ett och ända exemplet som CnAkE innehåller. En For Loop tar in tre argument: det initiala värdet, villkoret för när loopen ska avslutas och hur värdet ska ökas vid varje iteration.

Ex.

```
x=1;  
  
y=5;  
  
for_loop ( x; x < y ; x += 1 )  
print x;
```

Kodsnitt 4: Loopar

3.5 If Satser

If satser i CnAkE tar in ett argument och undersöker om det är sant eller falskt. Om den är sann, så uppfylls If satsen, och koden som tillhör satsen körs. Om den är falsk, så uppfylls inte kraven och koden som tillhör satsen körs inte.

Ex.

```
if (1 > 2) print 5; else print 9;
```

Kodsnitt 5: If Sats

3.6 Funktioner

Funktioner i CnAkE är en del av kod som kan kallas om och om igen i olika delar av koden som kan också mata in existerande variabler. Hur man skriver en funktion är att man börjar först med `func`, namn på funktionen, sedan variabler inom parantes om koden i funktionen behöver använda det, och sedan ett kolon. Efter det så skriver man koden som funktionen skall användas inom en indentering, och till slut avslutas med ett till kolon. Sedan för att använda funktionen så skriver man funktions namnet, och variabler om det behövs.

Ex.

```
func add ( a b ):  
    print a + b;  
;  
  
add ( 3 , 4 ) ;
```

Kodsnitt 6: Funktioner

4 Systemdokumentation

4.1 Beskrivning

CnAkE använder Ruby språket i interpretator läge för att köra och exekvera koden. Hur själva språket körs är med hjälp av "rdParse" som har använts under kursens gång och i en tidigare kurs vi har gått, TDP007. rdParse lexas och parsas i denna fil i en rekursiv stil. rdParse har blivit omnämnd till CnAkEparse i vårt sammanhang.

Sedan har vi ytterligare 'ast.rb' och 'CnAkElang.rb'. ast.rb innehåller vår logik inom våra AST noder som används. I CnAkElang.rb så innehåller den tokens och hur den matchar alla tokens med CnAkEparse.

4.2 BNF Grammatik

```
[program] ::= [statements]  
[statements] ::= [statement] | [simple_statement]
```

```

[simple_statement] ::= print | [expr]
[statement] ::= if [expr] | else_if [expr] | else | for_loop [expr][expr][expr] | func [vari]
[param_list] [statements] | print | [expr]
[param_list] ::= [param_list] [vari] | [vari]
[expr] ::= [type][vari]=[expr] | [vari]=[expr] | [expr]+[term] | [expr]-[term] | [expr]==[expr] |
[expr]!= [expr] | [expr]>=[expr] | [expr]<=[expr] | [expr]>[expr] | [expr]<[expr] | [term]+=[term]
| [term]-=[term] | [term]
[term] ::= [term]*[factor] | [term]/[factor] | [term]%[factor] | [vari]*=[term] | [vari]/=[term]
| [vari]%=[term] | [factor]
[factor] ::= [factor]**[atom] | [vari]**=[atom] | [atom]
[atom] ::= [vari][arg_list] | [vari] | Integer | String | Array | [expr]
[arg_list] ::= [arg_list][expr] | [expr] [vari] ::= Array
[type] ::= int | str | chr | pnt | bol

```

4.3 AST Grammatik

CnAkE använder sig av AST noder. AST noderna har information som byggs upp av olika variabler som kommer in genom parsern. Sedan körs evaluate på noderna inom Ruby för att sedan köra koden och samt spara variabler. Ett "env"-variabel skickas runt genom alla noder vilket innehåller variabler och annan information för de andra noderna så de kan veta vad som finns med.

4.3.1 ASTNode

ASTNode är bara en tom klass som gör det enkelt att skilja sig mellan stmt noderna i ast.rb filen för språket. Mest använd för noder som initialiserar variabler så de blir användbara.

4.3.2 Stmt

Stmt är bara en tom klass som gör det enkelt att skilja sig mellan ASTNode noderna i ast.rb filen för språket. Mest använd för If satserna, Loopar och enkla funktioner som Print inom språket.

4.3.3 IfStmt

IfStmt noden tar in tre variabler som är

```

@Condition
@then_branch
@else_branch

```

Dessa variabler körs först genom att kolla en If sats om villkoret (@Condition) uppfylls, då körs @then_branch med evaluate funktionen. Om villkoret inte uppfylls så körs @else_branch istället med evaluate.

4.3.4 ForStmt

ForStmt noden tar in fyra variabler som är

@Condition
@init_expr
@update_expr
@body

@init_expr körs först för att initialisera expression, sedan körs en while loop tills @condition inte uppfylls längre. I while loopen så uppdateras @body och expression.

4.3.5 FuncDefStmt

FuncDefStmt noden tar in tre variabler som är

@name
@params
@body

@name används för att spara namnet på funktionen så den kan hittas senare när den anropas. @params innehåller parametrarna som skickas in till funktionen. @body innehåller själva koden som ska köras när funktionen anropas. När funktionen definieras sparas den i programmets environment (env) så att andra delar av programmet kan använda funktionen senare.

4.3.6 FuncCallExpr

FuncCallExpr noden tar in två variabler som är

@name
@args

@name används för att hitta vilken funktion som ska köras och @args innehåller argumenten som skickas in till funktionen. När funktionen anropas hämtas först rätt funktion från environment. Sedan kopplas argumenten ihop med parametrarna från FuncDefStmt. Efter det körs funktionens body med evaluate. Detta gör att funktioner kan användas flera gånger med olika värden.

4.3.7 AssignStmt

AssignStmt noden har tre variabler

@name
@type
@value

AssignStmt används för att tilldela ett värde till en variabel, sen sedan spara variabeln i scope. @name är namnet till variabeln, @type är vilken typ variabeln är som en **int** eller **str** till exempel och @value är värdet som tilldelas till variabeln.

4.3.8 CompoundAssignStmt

CompoundAssignStmt noden tar in tre variabler som är

@name
@op
@value

CompoundAssignStmt nodentar in ett tal och beroende på operatoren tar den sitt och det nya värdet för att uppdatera sitt nuvarande värde.

4.3.9 PrintStmt

PrintStmt noden tar in en variabel som är

@expr

PrintStmt omvandlar @expr till en value som sedan skrivs ut direkt i terminalen.

4.3.10 ExprStmt

ExprStmt noden tar in en variabel som är

@Expr

ExprStmt helt enkelt kör en expression med evaluate så den kan användas senare som till exempel till aritmetiken i språket.

4.3.11 Program

Program noden tar in en variabel som är

@statements

När första matchningen körs så är det Program noden som börjar. Den kör sedan resten av alla statements som har parsats in och kör de andra AST noderna.

4.3.12 NumberLiteral

NumberLiteral noden tar in en variabel som är

@value

NumberLiteral AST noden skapar numret som en nod för språket.

4.3.13 VariabelRef

VariabelRef tar in en variabel som är

@name

VariabelRef AST noden refererar till variabeln som är nämnd i @name variabeln.

4.3.14 BinaryExpr

BinaryExpr noden tar in tre variabler

@op

@left

@right

@left och @right variablerna är de olika delarna i det binära uttrycket och @op används som operatoren för att få fram värdet som man har skrivit. Till exempel $1 + 2$ där 1 är @left, + är @op och 2 är @right.

4.4 Språkdelar

Vårt språk CnAkE är uppbyggt i flera distinkta delar som tillsammans möjliggör lexikal analys, parsing, och evaluering. Vi har valt att implementera språket med hjälp av biblioteket `rdParse`, vilket vi döpt om till `CnAkEparse` i vårt projekt.

- **Lexer (lexikalisk analysator):** Lexern ansvarar för att ta den råa strängen från användaren och omvandla den till tokens. Dessa tokens är exempelvis heltal, operatorer och identifierare som `int`, `chr` eller `print`. Lexern använder reguljära uttryck för att identifiera dessa.
- **Parser:** Parsningen görs med `rdParse` biblioteket, där vi definierar grammatiken för CnAkE. Grammatiken är uppdelad i olika parser regler, såsom för uttryck (`expr`), operationer (`op`), typer (`type`), variabelnamn (`vari`) och basfunktioner (`basf`). Dessa kombineras rekursivt till syntaxträd som används vid evaluering.
- **Evaluator:** Efter att ett syntaxträd har skapats används evalueringsklasser för att köra själva instruktionerna. Vi har implementerat evalueringslogik för bland annat aritmetiska uttryck, tilldelning, utskrift och variabelhantering. Alla evalueringsklasser ärver från en gemensam basstruktur som gör att vi enkelt kan lägga till fler funktioner som t.ex. funktioner eller loopar i framtiden.

4.5 Kodstandard

För att göra vår kod så läsbar och strukturerad som möjligt har vi följt kodstandard genom hela projektet:

- **Indentering och radbrytning:** Indentering görs med två mellanslag per nivå vilket är standard i Ruby och gör koden enkel att överblicka. Inga långa rader vi bryter långa uttryck för att säkra läsbarhet.

- **Kommentering:** Kommentarer används sparsamt men informativt för att förklara parseregler eller viktiga evalueringsklasser. Kommentarererna är skrivna på svenska för att följa projektkraven.
- **Felhantering:** Vi har börjat lägga till grundläggande felmeddelanden i parsningen, t.ex. om en datatyp saknas eller om syntaxen är fel. Dessa skrivs ut i terminalen för att hjälpa användaren förstå vad som gick fel.

Genom att följa denna kodstandard har vi säkerställt att vår kod är lätt att underhålla, utöka och läsa både för oss själva och för andra utvecklare som granskar projektet.

5 Programkod

Länken till git repositoryn : <https://gitlab.liu.se/emipe528/tdp019>

6 Erfarenheter och reflektion

Den största erfarenheten som vi har uppnått är såklart att veta hur ett programspråk faktiskt fungerar i praxis. Det har varit roligt och lärande att förstå hur allting fungerar bakom kulisserna och det har gett en uppskattning på hur allting är byggt upp. Vi har hört i andra kurser på hur andra programmerings språk är uppbyggda av andra språk, men själv visste vi inte hur det fungerade.

Men det hur vi skulle börja skapa språket var nog det svåraste. Vi hade en liten idé på hur vårt språk skulle se ut, men att faktiskt lägga det i kod, och sedan skapa ett språk med sin egen grammatik var klurigt. Så vi satt en bra tid med språket och kollade på andra språk på hur dem var uppbyggda. Sedan hade C ett eget systemdokumentation som man kunde kolla på och undersöka, vad som ingår i ett språk. De första handledningspassen var också väldigt hjälpsamma. Det var väldigt givande att höra på assistenten och att ställa frågor.

Med att vi inte visste hur vi skulle börja, och att hur öppet det här projektet var, så var det svårt att komma igång med att skriva i språket. Med seminarierna och tentan i TDP007 så var det all kunskap vi hade gått med, vilket inte var mycket. Vi visste hur variabler, aritmetik och sannings förhållanden skulle implementeras, men lite mer avancerade delar så stod vi helt stilla.